Verapay Technical Docs

Frontend Engineering Documentation

CLIENT ARCHITECTURE & COMPONENT SPECS • VERSION 2.0

Verapay Platform
Verify, then release.

Verapay: Frontend Engineering Documentation

Client Architecture, UI Component Specifications & Integration Guide | Version 2.0

1. Frontend Technology Stack & Architecture

Layer	Technology	Rationale
Framework	**Next.js 14+ (App Router)**	Hybrid SSR/SSG for rapid initial loads; API routes for edge authentication and webhook handling.
Language	**TypeScript 5.x**	Strict type safety for financial transactions, escrow states, and verification payloads.
Styling	**Tailwind CSS + shadcn/ui**	Rapid accessible component development; highly customizable for mobile-first views.
State Management	**Zustand + TanStack Query (v5)**	Zustand for local interactive state (camera, modal flows); TanStack Query for server state caching & optimistic UI.
Payments SDK	`@paypal/react-paypal-js`	Official React PayPal SDK for sandbox and production checkout buttons and subscription triggers.
Icons & Visuals	**Lucide React**	Feather-light SVG icons optimized for high-density mobile displays.
Media Handling	**Native WebRTC / MediaDevices API**	Direct browser camera and microphone streams; enforces live camera capture and blocks file pickers.

2. Design System & Design Tokens

Verapay uses a clean, high-trust fintech palette reminiscent of institutional security, with high-contrast accessibility across desktop and mobile screens.

```
:root {
  /* Brand Primary */
  --color-brand-primary: #007A5E;        /* Verapay Green */
  --color-brand-dark: #004D3A;           /* Evergreen for Headings */
  --color-brand-light: #E6F4EA;          /* Light Mint for Backgrounds */

  /* Semantic Traffic Light Signals */
  --signal-green-bg: #DCFCE7;            /* Green Alert Background */
  --signal-green-text: #15803D;          /* Green Text */
  --signal-green-border: #86EFAC;

  --signal-amber-bg: #FEF3C7;            /* Amber Alert Background */
  --signal-amber-text: #B45309;          /* Amber Text */
  --signal-amber-border: #FCD34D;

  --signal-red-bg: #FEE2E2;              /* Red Alert Background */
  --signal-red-text: #B91C1C;            /* Red Text */
  --signal-red-border: #FCA5A5;

  /* Neutrals */
  --color-surface: #FFFFFF;
  --color-surface-subtle: #F8FAFC;
  --color-text-main: #1E293B;
  --color-text-muted: #64748B;
  --color-border: #E2E8F0;
}
```

3. Core UI Component Specifications

3.1. `<TrafficLightDecisionCard />`

The primary decision interface presented to the sender on Gate 4. The sender sees a synthesized risk verdict with full forensic transparency.

```
// src/components/verification/TrafficLightDecisionCard.tsx

export type TrafficLightSignal = 'GREEN' | 'AMBER' | 'RED';

export interface VerificationCheckItem {
  id: string;
  label: string;
  passed: boolean;
  details: string;
}

export interface TrafficLightProps {
  paymentId: string;
  signal: TrafficLightSignal;
  headline: string;
  description: string;
  checks: VerificationCheckItem[];
  amountUSD: number;
  amountNGN: number;
  providerName: string;
  onApprove: (paymentId: string) => Promise<void>;
  onRefund: (paymentId: string) => Promise<void>;
  onViewAlternatives?: () => void;
  onRequestHumanReview?: () => void;
  isPlusUser: boolean;
}
```

UI Behavior & State Logic:

1. 🟢 **GREEN State:**

- Banner: Mint green background (`--signal-green-bg`) with checkmark shield icon.
- Actions: Primary CTA `[ Approve & Release $XXX ]` triggers `onApprove`. Secondary link: `[ Reject & Refund ]`.

2. 🟡 **AMBER State:**
   - Banner: Soft amber background (`--signal-amber-bg`) with alert triangle icon.
   - Prominently displays price delta: e.g., "*Cement quoted at ₦14,200/bag. Local verified suppliers average ₦10,800/bag.*"
   - Actions:
     - `[ View Cheaper Suppliers ]` opens `<QuoteComparisonModal />`.
     - `[ Request Human Review ]` (Free for Verapay Plus, with badge "*Reviewed in < 2 hrs*").
     - `[ Override & Pay Anyway ]` with explicit confirmation dialogue.
3. 🔴 **RED State:**
   - Banner: Muted red background (`--signal-red-bg`) with octagonal alert icon.
   - Forensic reason clearly stated: "*Photo reused from August 14th milestone*" or "*Beneficiary account name does not match school CAC records.*"
   - Actions: Prominent primary CTA `[ 1-Click Instant Refund to PayPal ]`. Payout button is strictly disabled.

3.2. `<InAppCamera />` (Anti-Spoof Live Viewfinder)

Ensures contractors and suppliers take photos live on site. Uploading existing images from the phone gallery is technically blocked.

```
// src/components/camera/InAppCamera.tsx

export interface CameraCapturePayload {
  blob: Blob;
  base64Data: string;
  metadata: {
    latitude: number;
    longitude: number;
    accuracyMeters: number;
    altitudeMeters: number | null;
    compassHeading: number | null;
    capturedAtISO: string;
  };
}

export interface InAppCameraProps {
  milestoneId: string;
  promptText: string;
  onCaptureComplete: (payload: CameraCapturePayload) => void;
  onCancel: () => void;
}
```

Technical Implementation Rules:

- **No File Input:** NEVER render `<input type="file">`. The component must directly call `navigator.mediaDevices.getUserMedia({ video: { facingMode: 'environment' } })`.
- **Geolocation Enforcement:** `navigator.geolocation.getCurrentPosition` is queried concurrently. If location services are disabled, camera feed displays a banner: "*Location access required to verify site coordinates.*"
- **Real-Time Canvas Watermark:** When the user taps the capture button, the frame is copied to an HTML5 `<canvas>`. Live GPS coordinates, timestamp, and active project ID are watermarked directly into the bottom-right pixels of the image buffer before encoding to JPEG.
- **Client-Side Compression:** Images are compressed to max 1920x1080 resolution at 0.75 JPEG quality before upload, reducing upload payload from ~8MB to ~450KB for fast uploads over Nigerian 3G/4G connections.

3.3. `<SurpriseVideoRecorder />` (15-Second Challenge)

Issues a randomized live video challenge to contractors to prove current physical site presence.

```
// src/components/camera/SurpriseVideoRecorder.tsx

export interface SurpriseVideoProps {
  milestoneId: string;
  challengeCode: string; // e.g. "CODE: 8291"
  instructions: string;  // e.g. "Pan north wall while holding challenge code"
  onUploadSuccess: (videoUrl: string) => void;
}
```

UX & MediaRecorder Architecture:

1. Modal displays challenge instructions and a big "Ready" button.
2. Once initiated, camera feed displays an active 15-second countdown timer and an on-screen badge showing the challenge code.
3. Uses `MediaRecorder` API (`video/webm;codecs=vp8,opus` or `video/mp4` on Safari/iOS).
4. Automatically halts recording at 15.0 seconds.
5. Displays immediate playback preview with `[ Submit Video Proof ]` or `[ Re-record ]`.

3.4. `<MilestoneProgressStepper />`

Used in Verapay Projects to visualize capital construction phases.

```
// src/components/projects/MilestoneProgressStepper.tsx

export interface MilestoneStage {
  id: string;
  sequenceNumber: number;
  title: string;          // e.g. "Foundation & German Floor"
  targetBudgetUSD: number;
  status: 'LOCKED' | 'READY_TO_FUND' | 'IN_PROGRESS' | 'IN_VERIFICATION' | 'APPROVED';
  proofSummary?: {
    photoCount: number;
    videoVerified: boolean;
    surveyorVisited: boolean;
  };
}
```

3.5. `<QuoteComparisonModal />`

Autonomous Quote Agent interface showing contractor price vs. local market quotes.

```
// src/components/verification/QuoteComparisonModal.tsx

export interface SupplierQuote {
  supplierId: string;
  businessName: string;
  location: string;
  pricePerUnit: number;
  totalCost: number;
  cacVerified: boolean;
  phone: string;
}

export interface QuoteComparisonProps {
  materialName: string;      // e.g. "Dangote 42.5R Cement (500 Bags)"
  contractorPrice: number;   // e.g. ₦7,100,000 (₦14,200 / bag)
  marketAveragePrice: number;// e.g. ₦5,400,000 (₦10,800 / bag)
  alternativeSuppliers: SupplierQuote[];
  onSelectAlternative: (supplier: SupplierQuote) => void;
  onClose: () => void;
}
```

3.6. `<SharedPotWidget />` & `<FamilyGroupDashboard />`

The collaborative hub for Verapay Plus subscribers.

```
// src/components/family/SharedPotWidget.tsx

export interface PotContributor {
  userId: string;
  name: string;
  avatarUrl: string;
  amountContributedUSD: number;
  fundedAtISO: string;
}

export interface SharedPotProps {
  potId: string;
  potTitle: string;          // e.g. "Dad's Eye Surgery at LUTH"
  targetAmountUSD: number;
  totalCollectedUSD: number;
  contributors: PotContributor[];
  status: 'COLLECTING' | 'TARGET_REACHED' | 'DISBURSED';
  onChipIn: () => void;
}
```

4. Application Routing & Screen Flow Map

```
/app
 ├── (auth)
 │    ├── /login                    # Email OTP / Google / PayPal Auth
 │    └── /onboarding               # Sender or Beneficiary profile setup
 ├── (dashboard)
 │    ├── /dashboard                # Overview of active escrows & monthly spend
 │    ├── /pay/new                  # Payee search & bill upload flow
 │    ├── /pay/[id]/verify          # Gate 4 Traffic Light Decision Screen
 │    ├── /family                   # Verapay Plus Family Group & Shared Pots
 │    │    └── /pots/[id]           # Shared Pot funding & contribution view
 │    └── /projects                 # Verapay Projects list
 │         ├── /projects/new        # Initialize stage-by-stage construction
 │         └── /projects/[id]       # Milestone Stepper & Progress Tracker
 └── (mobile-contractor)
      ├── /contractor/upload/[token] # Magic link for contractor on-site photo upload
      └── /beneficiary/confirm/[id]  # 1-tap "Item Received" confirmation sheet
```

5. Client State Management Architecture

```
// src/stores/usePaymentStore.ts
import { create } from 'zustand';

interface PaymentState {
  activePaymentId: string | null;
  currentStep: number;
  invoiceFile: File | null;
  ocrExtractedData: any | null;
  trafficLightVerdict: 'GREEN' | 'AMBER' | 'RED' | null;
  setActivePayment: (id: string) => void;
  setOcrData: (data: any) => void;
  setVerdict: (verdict: 'GREEN' | 'AMBER' | 'RED') => void;
  reset: () => void;
}

export const usePaymentStore = create<PaymentState>((set) => ({
  activePaymentId: null,
  currentStep: 1,
  invoiceFile: null,
  ocrExtractedData: null,
  trafficLightVerdict: null,
  setActivePayment: (id) => set({ activePaymentId: id }),
  setOcrData: (data) => set({ ocrExtractedData: data }),
  setVerdict: (verdict) => set({ trafficLightVerdict: verdict }),
  reset: () => set({ activePaymentId: null, currentStep: 1, invoiceFile: null, trafficLightVerdict: null }),
}));
```

6. PayPal Client SDK Integration

```
// src/components/payments/PayPalEscrowButton.tsx
import React from 'react';
import { PayPalButtons, usePayPalScriptReducer } from '@paypal/react-paypal-js';

interface PayPalEscrowButtonProps {
  amountUSD: number;
  paymentId: string;
  onFundingSuccess: (orderId: string) => void;
  onFundingError: (error: any) => void;
}

export const PayPalEscrowButton: React.FC<PayPalEscrowButtonProps> = ({
  amountUSD,
  paymentId,
  onFundingSuccess,
  onFundingError,
}) => {
  const [{ isPending }] = usePayPalScriptReducer();

  if (isPending) {
    return <div className="h-12 w-full animate-pulse bg-gray-200 rounded-lg" />;
  }

  return (
    <PayPalButtons
      style={{ layout: 'vertical', shape: 'rect', color: 'gold' }}
      createOrder={async () => {
        const response = await fetch(`/api/v1/payments/${paymentId}/create-paypal-order`, {
          method: 'POST',
        });
        const order = await response.json();
        return order.id;
      }}
      onApprove={async (data) => {
        const response = await fetch(`/api/v1/payments/${paymentId}/capture-escrow`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ orderId: data.orderID }),
        });
        const result = await response.json();
        if (result.success) {
          onFundingSuccess(data.orderID);
        } else {
          onFundingError(result.error);
        }
      }}
      onError={(err) => onFundingError(err)}
    />
  );
};
```

7. Nigerian Network & Low-Bandwidth Optimizations

1. **Aggressive Image Downsampling:** Mobile WebRTC canvas scales photos to 1920x1080 and compresses to JPEG 75% prior to transmission.
2. **Chunked Video Streaming:** 15-second video uploads use chunked HTTP POST requests via `Blob.slice()` with automatic exponential backoff retry.
3. **Optimistic Local Caching:** TanStack Query `staleTime` set to 10 minutes for CAC verified directories and quote indices.
4. **Service Worker Offline Cache:** Static assets, icons, and shell layouts are pre-cached via Workbox, allowing smooth UI rendering even on intermittent Nigerian 3G connections.